

CSE 203: Data Structures

Lecture 4: Elementary Data Structures- Queue

Queues

FIFO: First in, First Out

Restricted form of list: Insert at one end, remove from the other.

Notation:

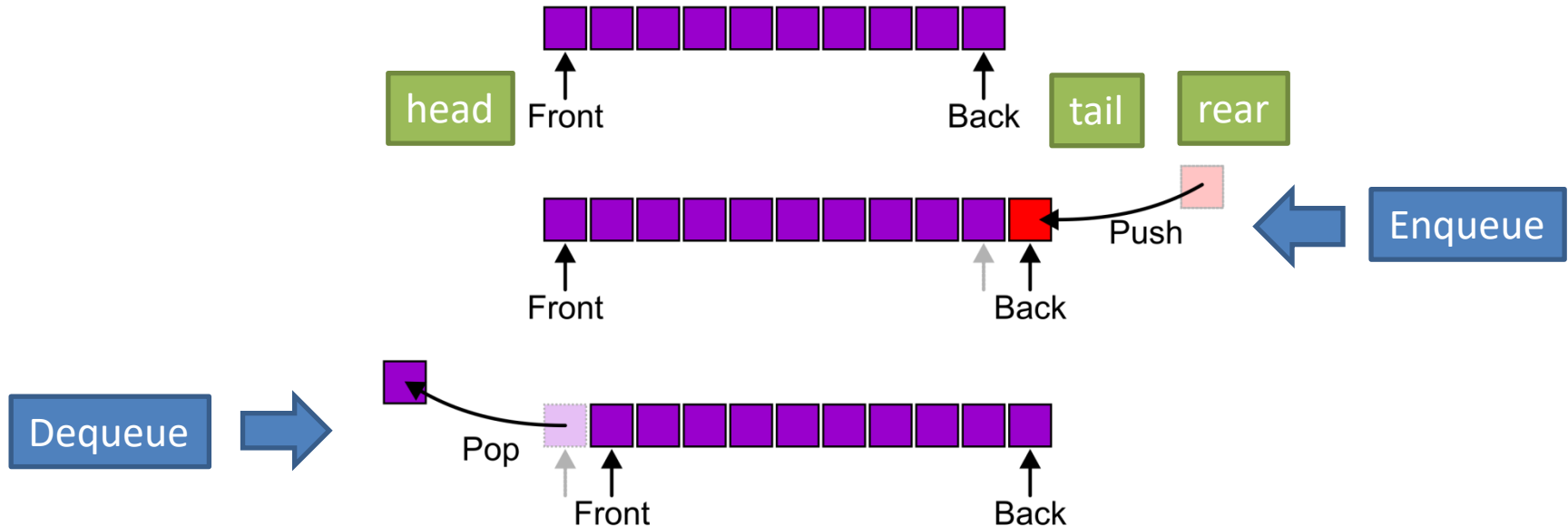
- Insert: Enqueue
- Delete: Dequeue
- First element: Front
- Last element: Rear



Abstract Queue

Also called a *first-in–first-out* (FIFO) data structure

- Graphically, we may view these operations as follows:



Abstract Queue

There are two exceptions associated with abstract Queue data structure:

- It is an undefined operation to call either pop or front on an empty queue

Queue ADT in C++



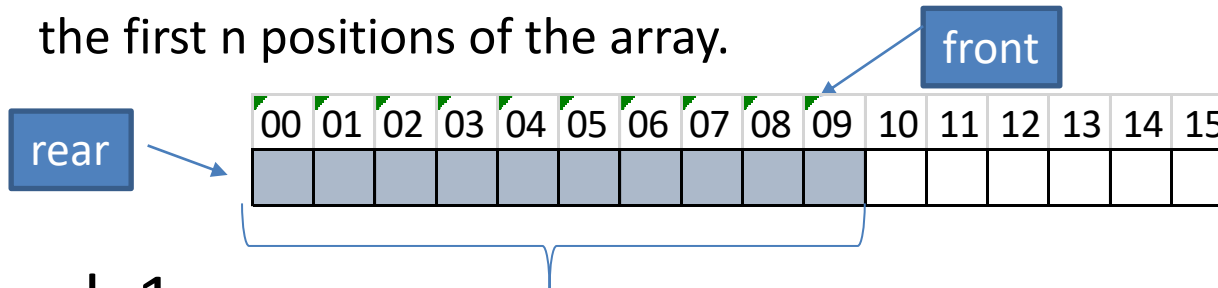
C++3elatestP129.pdf

Array Based Implementation

- The array-based queue is somewhat tricky to implement effectively.
- A simple conversion of the array-based list implementation is not efficient.
- Why?

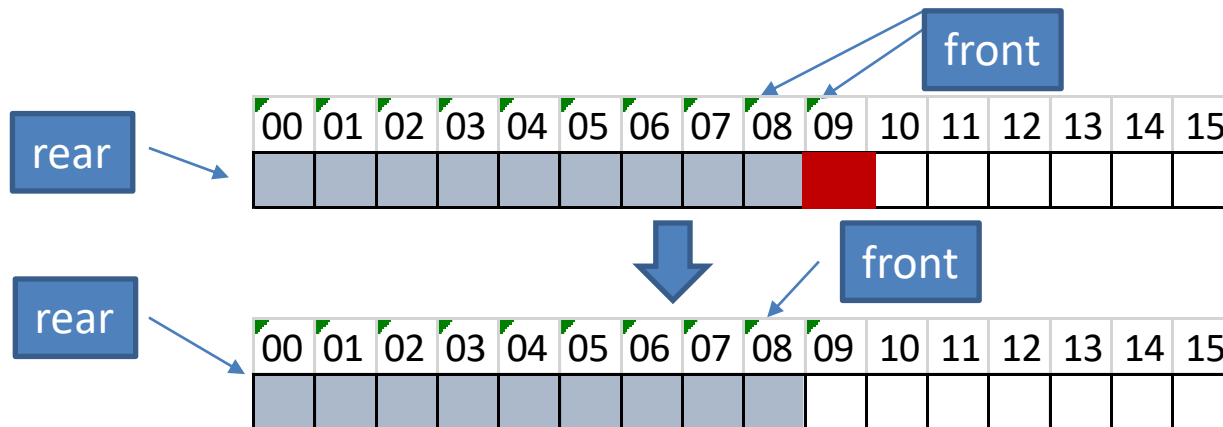
Array Based Implementation

- Assume that there are n elements in the queue and Like our list implementation, we require that all elements of the queue be stored in the first n positions of the array.



Impl. 1:

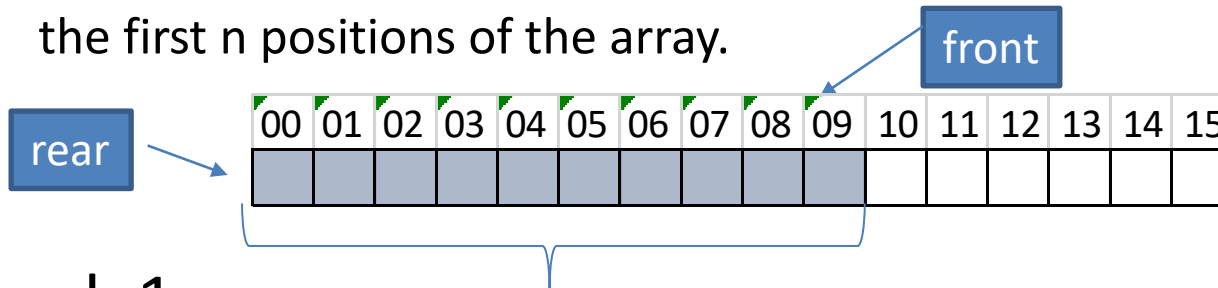
- The rear element of the queue is at position 0,



- **dequeue operations require only $O(1)$ time because the front element of the queue is the last element in the array.**

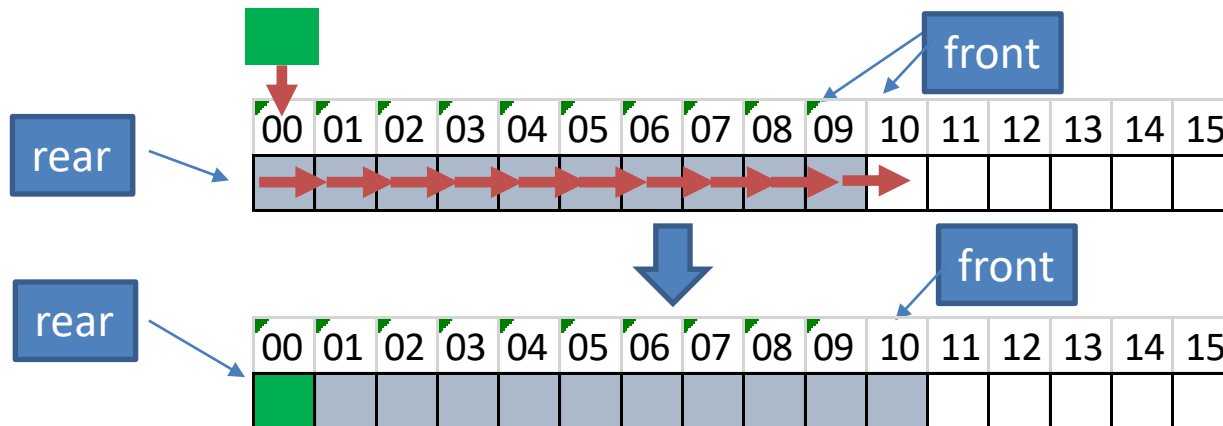
Array Based Implementation

- Assume that there are n elements in the queue and Like our list implementation, we require that all elements of the queue be stored in the first n positions of the array.



Impl. 1:

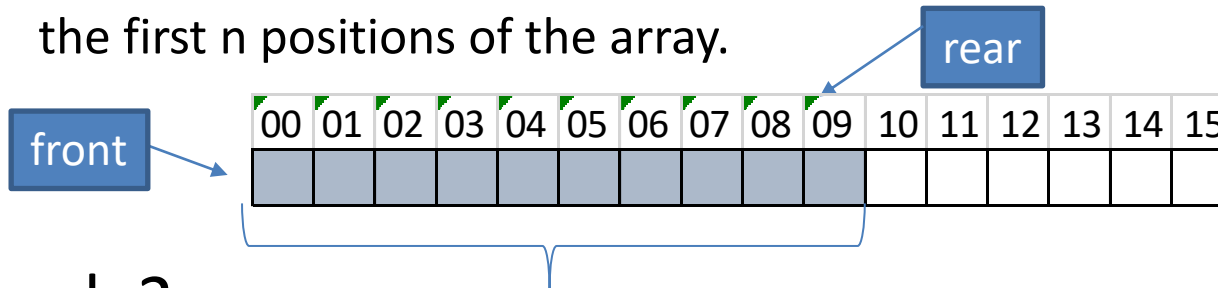
- The rear element of the queue is at position 0,



- However, **enqueue operations will require $O(n)$ time, because the n elements currently in the queue must each be shifted back.**

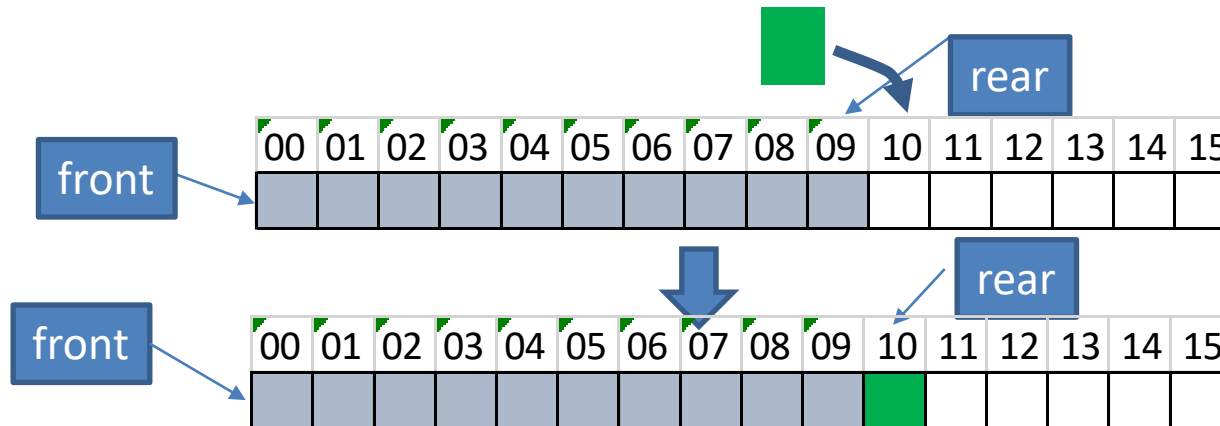
Array Based Implementation

- Assume that there are n elements in the queue and Like our list implementation, we require that all elements of the queue be stored in the first n positions of the array.



Impl. 2:

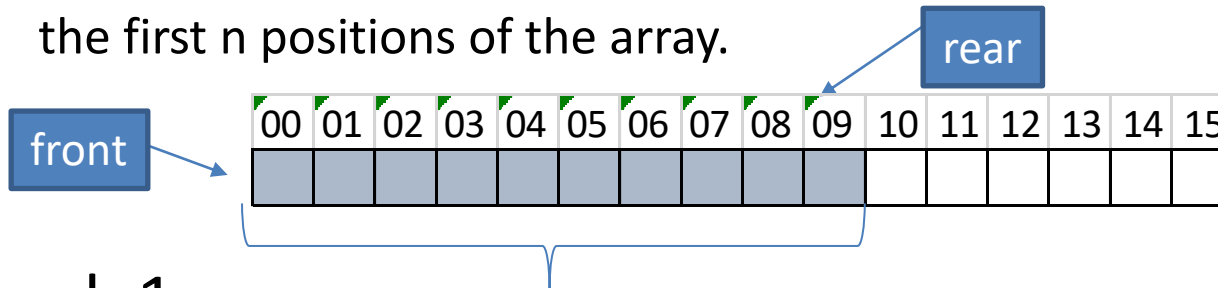
- The rear element of the queue is at position $n-1$,



- then an **enqueue operation** is equivalent to an **append operation** on a list. This requires only **$O(1)$** time.

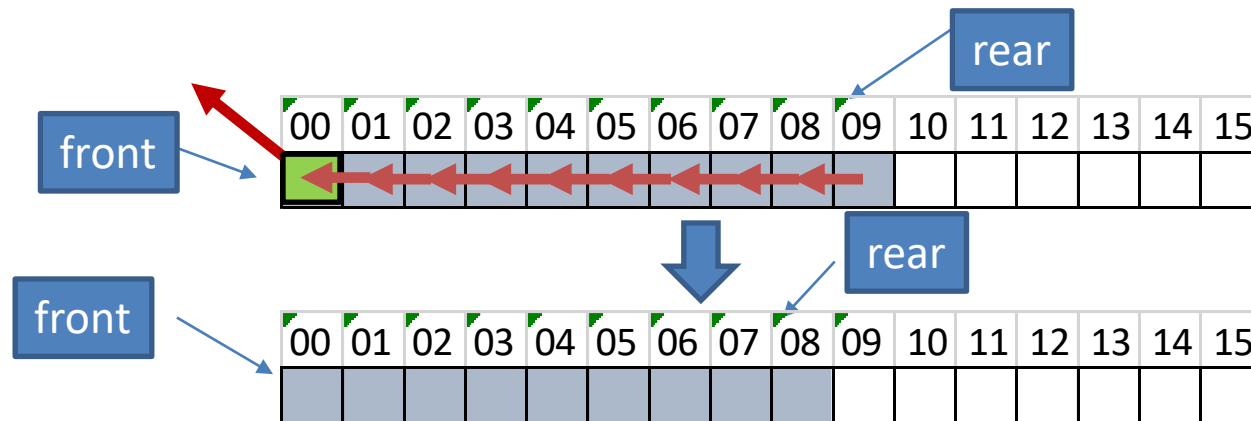
Array Based Implementation

- Assume that there are n elements in the queue and Like our list implementation, we require that all elements of the queue be stored in the first n positions of the array.



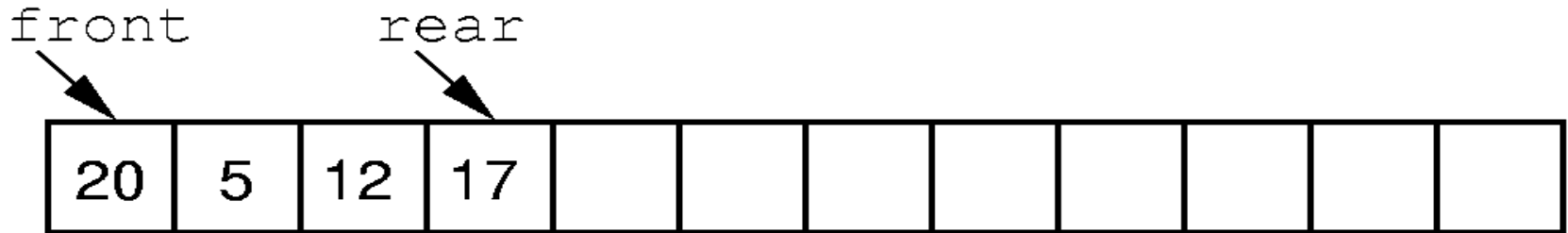
Impl. 1:

- The rear element of the queue is at position 0,

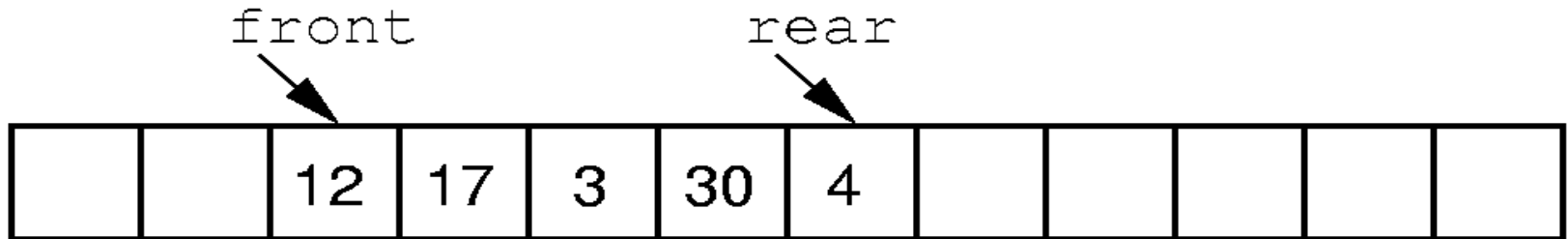


- But now, a **dequeue operation requires $O(n)$ time, because all of the elements must be shifted down by one position.**

Solution: Queue Implementation (1)



(a)



(b)

(a) Gives the Queue after 20, 5, 12, 17 are enqueued initially. (b) Gives Queue after 20 and 5 are dequeued and 3, 30, 4 are enqueued.

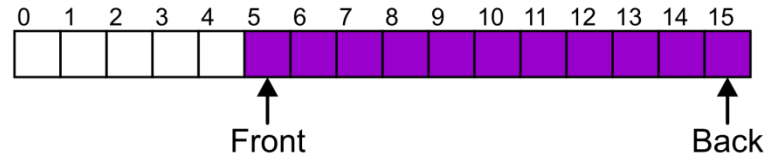
Performance

- Enqueue: $O(1)$
- Dequeue: $O(1)$
- Drifting Queue problem:
 - Dequeue $\Rightarrow$ the front index increases.
 - Over time, the entire queue will drift toward the higher-numbered positions in the array.
 - Once an element is inserted into the highest-numbered position in the array, the queue has run out of space.
 - This happens despite the fact that there might be free positions at the low end of the array where elements have previously been removed from the queue.

Drifting Queue Problem

Suppose that:

- The array capacity is 16
- We have performed 16 pushes
- We have performed 5 pops
 - The queue size is now 11



- We perform one further push

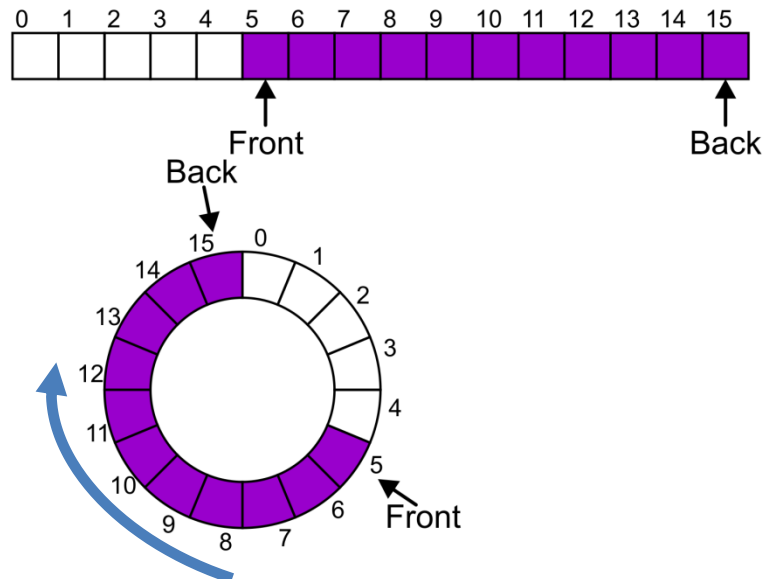
In this case, the array is not full and yet we cannot place any more objects in to the array

Solution to Drifting Queue Problem: Implementation (2)

Instead of viewing the array on the range $0, \dots, 15$, consider the indices being cyclic:

$\dots, 15, 0, 1, \dots, 15, 0, 1, \dots, 15, 0, 1, \dots$

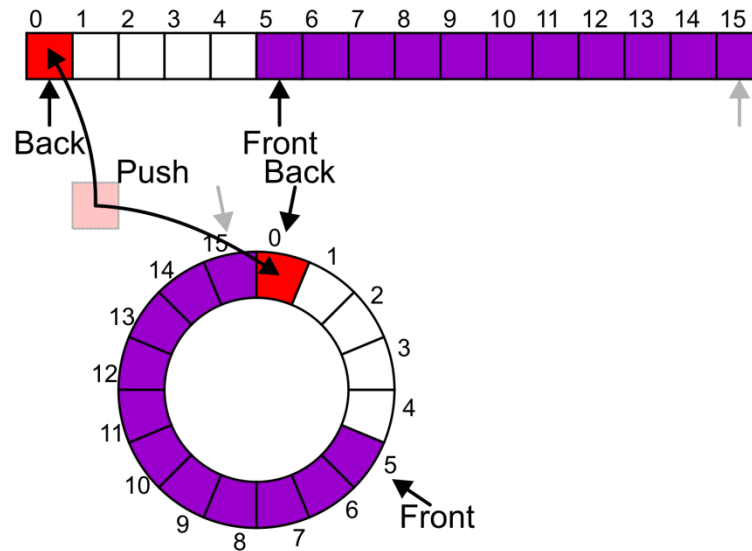
This is referred to as a *circular array*



Solution to Drifting Queue Problem: Implementation (2)

Now, the next push may be performed in the next available location of the circular array:

push => enqueue

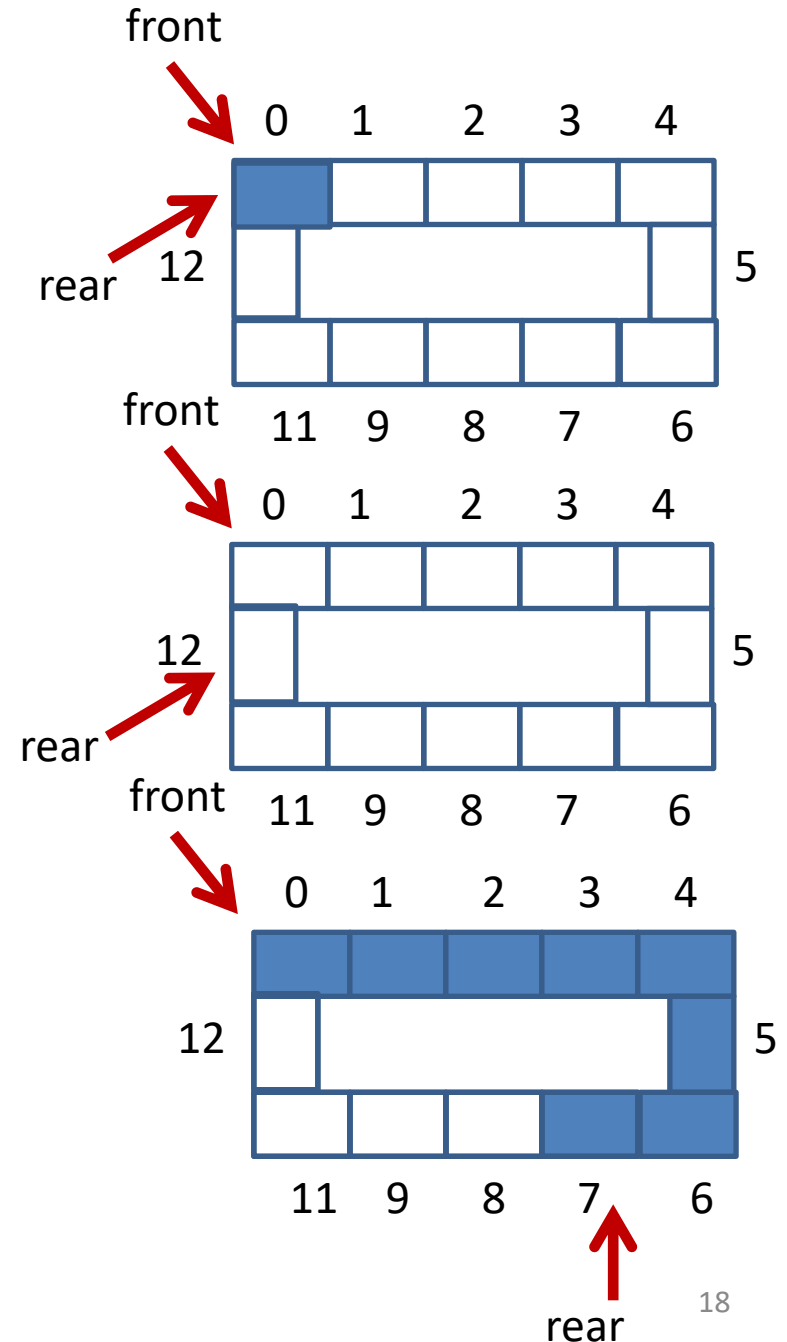
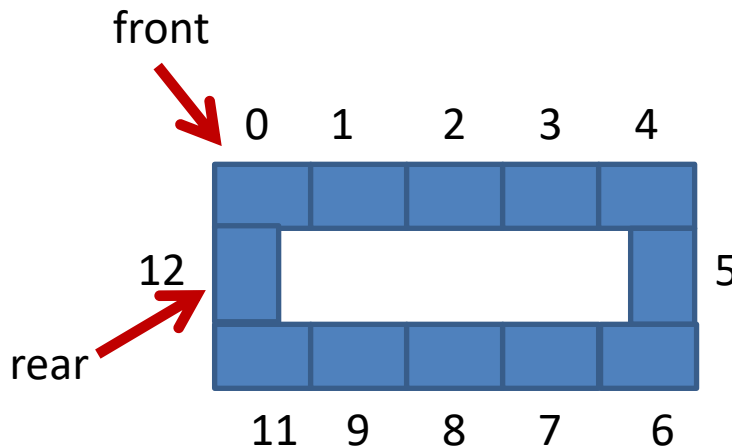


Implementation

- We allow the queue to continue directly from the highest-numbered position in the array to the lowest-numbered position.
- This is easily implemented through use of the modulus operator (denoted by % in C++).
 - **positions** in the array are numbered from 0 through **size-1**,
 - **and position size-1 is defined** to immediately precede position 0
 - (which is equivalent to position **size % size**).

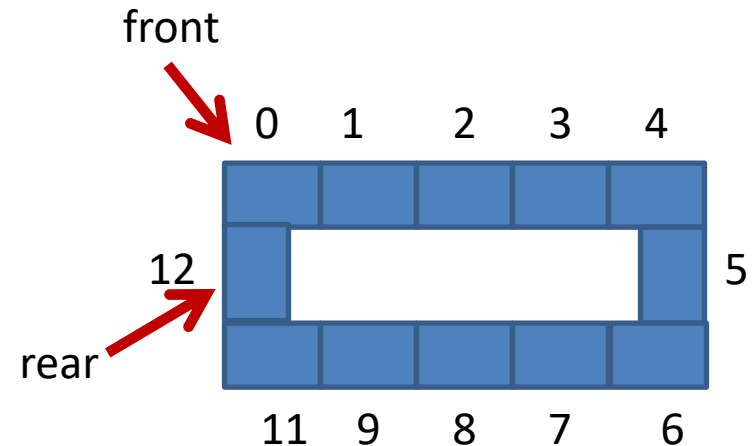
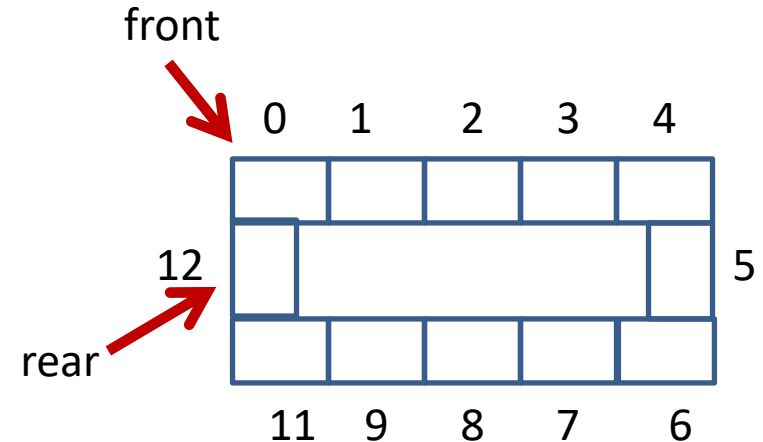
One Subtle Issue

- How can we recognize when the queue is empty or full?
 - $\text{front} = \text{rear}$?
 - This means one element is there.
 - $\text{rear} = \text{front} - 1$?
 - Seems a solution.
 - What happens when the queue is full?



A strange proof

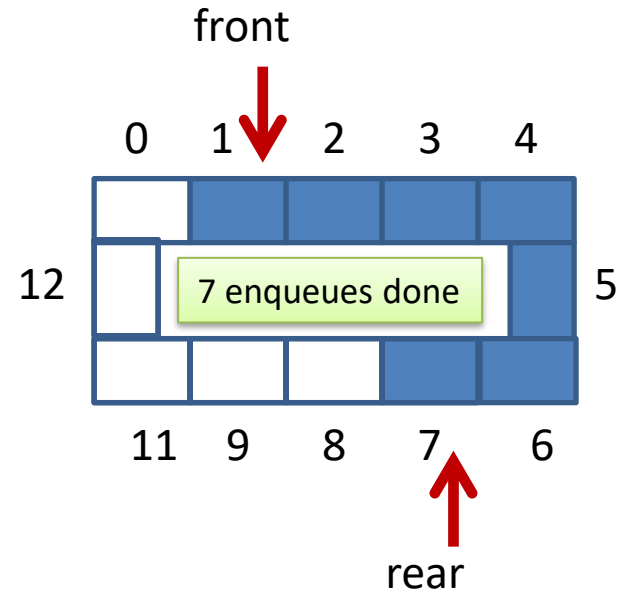
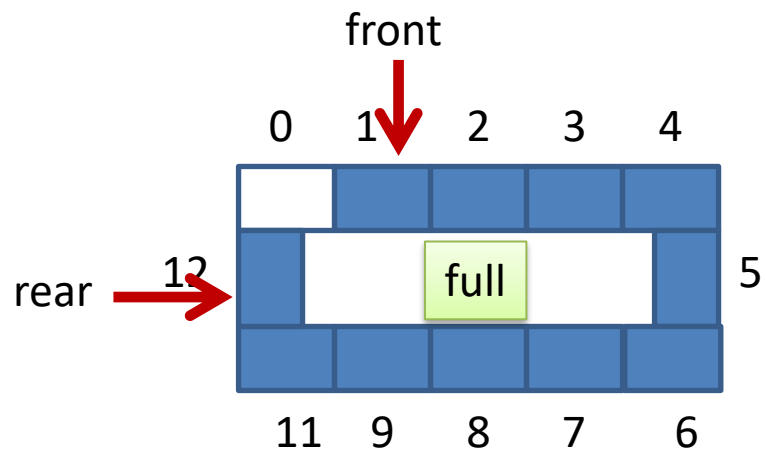
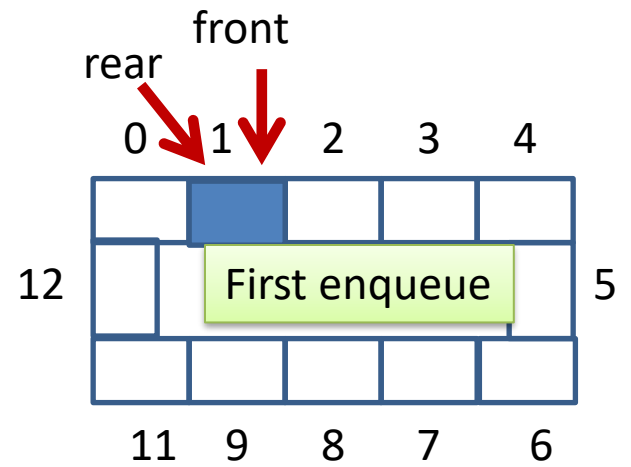
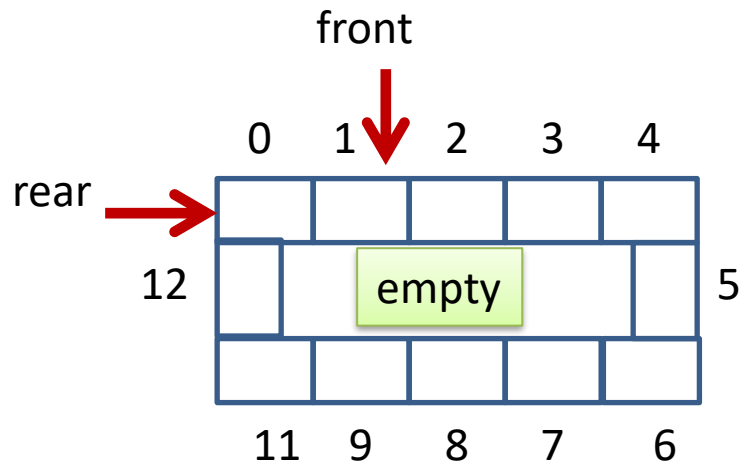
- We can in fact prove that we cannot solve the above issue by changing the definition of front and rear
- Assume front is fixed.
- We can have n values of rear
- But we can have $n+1$ possible states of the queue
- Pigeon hole principle $\Rightarrow$ one value of rear must be mapped to 2 different states!!!



2 different states for
the same value of rear

Two Solutions

- keep an explicit count of the number of elements in the queue, or at least a Boolean variable that indicates whether the queue is empty or not.
- Another solution is to make the array be of size $n + 1$, and only allow n elements to be stored.
- We use the second one...



Array implementation of Queue



C++3elatestP132.pdf

Linked List Implementation of Queue

- We will use a header link node, which allows for a simpler implementation of the enqueue operation by avoiding any special cases when the queue is empty.
- On initialization, the **front** and **rear pointers will point to the header node, and front will always point to** the header node while rear points to the true last link node in the queue.
- Method **enqueue places the new element in a link node at the end of the linked list (i.e., the node that rear points to) and then advances rear to point to the new link node.**
- Method **dequeue removes and returns the first element of the list.**

Linked List Implementation of Queue

- There is a small mistake in clear()
- `rear = front => rear = front-next`



C++3elatestP134.pdf